

Git Cheat sheet

Quick Start: How to Use Git

What is Git?

Git is a version control system that tracks changes in your code, allowing you to collaborate with others, maintain a history of changes, and manage different versions of your project.

Why Use Branches?

Branches allow you to work on different features or experiments without affecting the main codebase. Think of branches as parallel universes for your code:

- **main/master branch:** The stable, production-ready code
- **feature branches:** Where you develop new features or fix bugs
- **hotfix branches:** For urgent fixes that need to go to production quickly

Basic Workflow for Team Collaboration

1. **Clone the repository:** Get a copy of the project on your machine
2. **Create a branch:** Make a new branch for your feature or bugfix
3. **Make changes:** Edit files and commit your changes regularly
4. **Push your branch:** Upload your branch to the remote repository
5. **Create a pull request:** Ask teammates to review and merge your changes
6. **Merge and cleanup:** Once approved, merge into main and delete the feature branch

Essential Workflow Commands

```
# Start working on a new feature
git checkout main# Switch to main branch
git pull origin main# Get latest changes
git checkout -b feature-login# Create and switch to new branch# Work and commit
```

```
git add .# Stage your changes
git commit -m "Add login form"# Commit with descriptive message# Share your work
git push origin feature-login# Push branch to remote# Create pull request on GitHub/GitLab# After review and merge:
git checkout main# Switch back to main
git pull origin main# Get the merged changes
git branch -d feature-login# Delete local feature branch
```

Getting Started

```
git init# Initialize a new Git repository
git clone <url># Clone a repository
```

Staging and Changes

```
git status# View current changes
git add <file># Stage specific file
git add .# Stage all changes
git reset <file># Unstage a file
git diff# View unstaged changes
git diff --staged# View staged changes
```

Committing

```
git commit -m "message" # Commit with message
git commit --amend # Edit last commit
git commit -a -m "message" # Stage & commit tracked files
```

Branching

```
git branch# List branches
git branch <name># Create new branch
```

```
git checkout <name># Switch to branch
git checkout -b <name># Create and switch
git merge <branch># Merge another branch into current
git branch -d <name># Delete a branch
```

Working with Remotes

```
git remote -v# List remotes
git remote add origin <url># Add remote repository
git push origin <branch># Push changes
git pull origin <branch># Pull changes
git fetch # Fetch updates without merging
```

Reset, Revert, and Clean

```
git reset --soft HEAD~1# Undo commit, keep changes staged
git reset --mixed HEAD~1# Undo commit, unstage changes
git reset --hard HEAD~1# Discard commit and all changes
git revert <commit># Create new commit to undo previous
git clean -fd# Remove untracked files and folders
```

History and Logs

```
git log# Commit history
git log --oneline# Condensed log
git log --graph --all# Visualize branches
git blame <file># See who last changed each line
git show <commit># Show specific commit details
```

Git Commit Etiquette

Good commit messages and habits make collaboration easier, improve code history readability, and help with debugging.

Writing Good Commit Messages

- Use the **imperative mood**: "Fix typo in README" (not "Fixed" or "Fixes")
- Keep the subject line ≤ 50 characters
- Add a blank line after the subject
- Wrap the body at ~ 72 characters
- In the body, explain **what** and **why**, not **how**

Best Practices

- Make commits **small and focused** — one logical change per commit
- Use **Conventional Commits** prefixes to provide a clear and structured history:

Prefix	Purpose
feat	A new feature for the user
fix	A bug fix for the user
docs	Changes to documentation
style	Formatting changes (no logic change)
refactor	Code change that isn't a bug fix or feature
perf	Performance improvements
test	Adding or correcting tests
chore	Build process or auxiliary tool changes

- **Test** before committing when possible
- Use a `.gitignore` file to avoid committing unwanted files

Advanced: Using a Commit Footer

Use footers to provide context for tracking systems or breaking changes.

Referencing Issues

Link commits to issue trackers:

```
Fixes #42
Closes JIRA-123
Indicating Breaking Changes
```

BREAKING CHANGE: The 'userId' field is now a string, not an integer.

Example of a Full Commit Message

feat(api): Add pagination to article list view

Allows users to load more articles without refreshing the page.
Implements offset-based pagination on the API and UI.

Fixes #27

What to Avoid

- "WIP" or "misc changes" as commit messages
- Large, unrelated changes in a single commit
- Mixing formatting changes with logic changes
- Committing temporary/debug code or commented-out blocks